

智能算法及应用

11. 策略学习方法

智能算法及应用@华南理工大学2025

策略学习

不同于值学习方法, 策略梯度方法通常用某种方式来直接表示策略 π 。考虑在某个特定的策略下, 如果能够通过随机采样无数个Trajectories: $(\tau_1, \tau_2, \tau_3 \dots\dots)$, 那我们就可以估计这个策略带来的回报水平, 可以将其表示为:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi}[R(\tau)]$$

其中 τ 是采样的轨迹, $R(\tau) = \sum_{t=1}^T R_t$ 是有限步 (适用于多数情形) 的奖励。在这样的思考角度下, 不难发现我们其实可以通过在所有可行策略组成的空间 Π 中去进行搜索, 找到可以使上述期望最大的 π^* 。这种思路就是策略梯度方法和值学习方法的本质区别。

策略学习

那么怎么通过学习的方式去搜索 π^* 呢？最直接的办法就是参数化 π ，如我们可以使用一个神经网络来参数化策略 $\pi_{\theta}(a | s)$ ，然后寻找一个最优的参数 θ^* ，使基于该策略的累计回报的期望最大。



我们巧妙地将在 Π 空间的搜索任务转换到了一个连续的实数线性空间 Θ (神经网络参数的空间) 中的搜索任务，在这个空间我们可以用的手段很多，比如在这里我们是神经网络，就可以使用梯度下降/上升和反向传播。

策略梯度的推导

$$\max J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)] = \int p_{\theta}(\tau) R(\tau) d\tau, \text{ s.t. } \theta \in \Theta$$

对于一个 Trajectory $\tau : (s_1, a_1, r_1, \dots, s_T, a_T, r_T, s_{T+1})$

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

直接代入存在两个问题：

1. 乘积的导数计算十分复杂！
2. 在Model-Free方法中状态转移函数不可知！

策略梯度的推导

$$\max J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)] = \int p_{\theta}(\tau) R(\tau) d\tau, \text{ s.t. } \theta \in \Theta$$



1.

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \{ \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) \} d\tau$$

- 第1个等式成立是因为求导对象与积分对象不同
- 第2个等式利用了对数-导数技巧 (Log-Derivative Trick):
 $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$.

策略梯度的推导

$$\max J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)] = \int p_{\theta}(\tau) R(\tau) d\tau, \text{ s.t. } \theta \in \Theta$$



1.

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \{ \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) \} d\tau$$

将上式最后括号括起来的部分看作一个统计变量, 那么我们可以将梯度又写回期望的形式:

2.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

策略梯度的推导

对于一个 Trajectory $\tau : (s_1, a_1, r_1 \dots, s_T, a_T, r_T, s_{T+1})$

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

智能算法及应用@华南理工大学2025



$$\begin{aligned} \nabla_{\theta} \log p_{\theta}(\tau) &= \nabla_{\theta} \log \left[p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t) \right] \\ &= \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad \text{代入2.} \end{aligned}$$

策略梯度的推导

最终得到简化后的梯度形式为：

3.
$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left(\sum_{t=1}^T r(s_t, a_t) \right) \right]$$

智能算法及应用@华南理工大学2025

对 τ 进行采样, 以蒙特卡洛的方式来近似期望：

4.
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right) \right]$$

策略梯度的推导

然后便可用梯度上升方法更新参数 θ 值。

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

智能算法及应用@华南理工大学2025

REINFORCE算法

REINFORCE algorithm:

- 
1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
 2. $\nabla_\theta J(\theta) \approx \sum_i (\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i)) (\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i))$
 3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

改进一：修正回报

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right) \right]$$

- 可以把第 i 条序列的累积奖励 $\sum_{t=1}^T r(s_t^i, a_t^i)$ 项看作对数概率 $\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$ 的权重
- 对于序列中的每一个时间段的元组 (s_t^i, a_t^i) 的权重都是总回报值，理论上这是不合理的， t 时刻所做的动作只能影响该时刻之后的回报，而不应该影响该时刻之前的。

改进一：修正回报

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right) \right]$$



智能算法及应用@华南理工大学2025

因此，通常将权重项变为 $\sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}^i, a_{t'}^i)$ ，该式表示只计算时刻 t 之后的奖励，并且引入衰减系数 γ 。于是可将梯度公式写为：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \left[\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}^i, a_{t'}^i) \right]$$

改进二：引入基线

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \left[\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \sum_{t'=t}^T \gamma^{t'-t} r(s_t^i, a_t^i) \right]$$

- 奖励的随机性可能随着序列的长度呈指数级增长，导致训练时有较大的方差，效果不稳定。
- $\sum_{t'=t}^T \gamma^{t'-t} r(s_t^i, a_t^i)$ 可能始终为正，也就是会导致所有策略都会增强。而我们的初衷是降低表现差的动作的概率，提升表现好的动作的概率。

最简单的解决方式叫做“**Baseline**”。其基本想法是采样到的累积奖励减去一个基线值，作为“**Advantage**”

改进二：引入基线

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \left[\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \sum_{t'=t}^T \gamma^{t'-t} r(s_t^i, a_t^i) \right]$$



智能算法及应用@华南理工大学2025

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T [\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \cdot (R(\tau^i) - b)]$$

$$b = \frac{1}{N} \sum_{i=1}^N R(\tau^i)$$

改进二：引入基线

我们通过证明来说明 Baseline 这种做法不会对于期望的估计产生影响，可以看到不论 Baseline 取什么值，下式都成立。

$$\begin{aligned}\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau)b] &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0\end{aligned}$$

这意味着：

$$\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau)(R(\tau) - b)] = \mathbb{E}[\nabla_{\theta} \log_{\theta}(\tau)R(\tau)]$$

Actor-Critic

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \left[\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) \sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}^i, a_{t'}^i) \right]$$

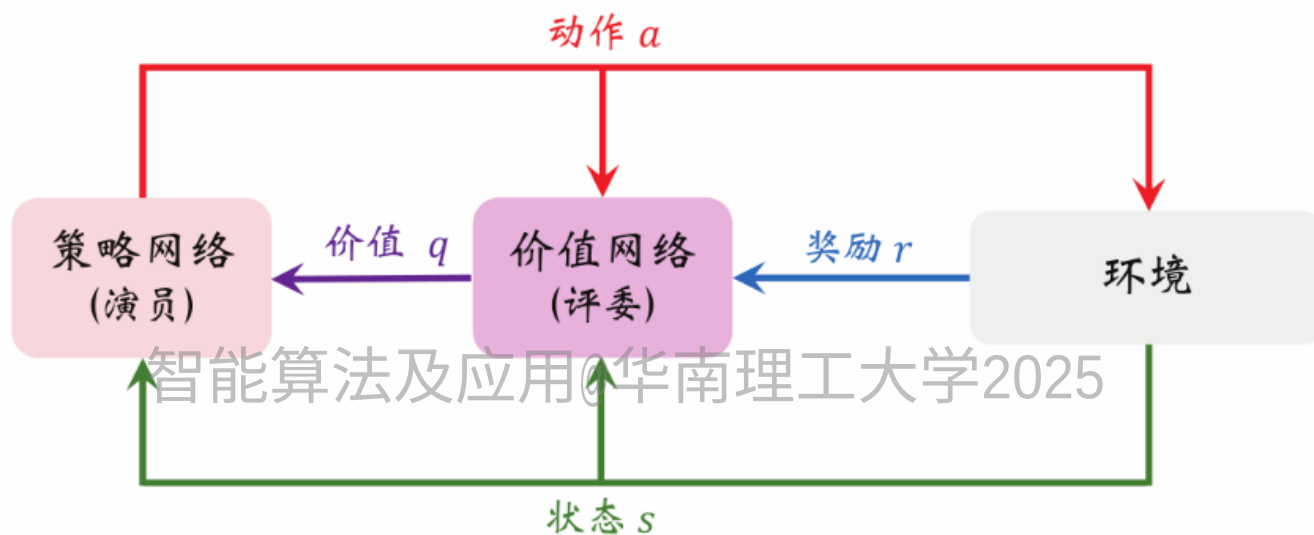
引入值函数Q

智能算法及应用@华南理工大学2025

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T [\nabla_{\theta} \log \pi_{\theta} (a_t^i | s_t^i) Q_{\pi} (s_t^i, a_t^i)]$$

- 前述REINFORCE方法即用蒙特卡洛的方式对状态动作价值函数进行了估计。优点是无偏性，缺点是方差大。尽管采用Baseline方法能从一定程度上减小方差，但因为每次采样的数据有限，方差依然很大，训练过程依然不稳定。
- 我们自然而然地想到另一类方法来估计Q值，即时间差分(TD)方法。

Actor-Critic



- 策略网络 $\pi_{\theta}(a | s)$ 相当于演员, 它基于状态 s 做出动作 a 。
- 价值网络 $Q_w(s, a)$ 相当于评委, 它给演员的表现打分, 量化在状态 s 的情况下做出动作 a 的好坏程度。

Actor-Critic

Algorithm 1 Q Actor Critic

Initialize parameters s, θ, w and learning rates α_θ, α_w ; sample $a \sim \pi_\theta(a|s)$.

for $t = 1 \dots T$: **do**

 Sample reward $r_t \sim R(s, a)$ and next state $s' \sim P(s'|s, a)$

 Then sample the next action $a' \sim \pi_\theta(a'|s')$

 Update the policy parameters: $\theta \leftarrow \theta + \alpha_\theta Q_w(s, a) \nabla_\theta \log \pi_\theta(a|s)$; Compute the correction (TD error) for action-value at time t:

$$\delta_t = r_t + \gamma Q_w(s', a') - Q_w(s, a)$$

 and use it to update the parameters of Q function:

$$w \leftarrow w + \alpha_w \delta_t \nabla_w Q_w(s, a)$$

 Move to $a \leftarrow a'$ and $s \leftarrow s'$

end for

Actor-Critic

我们可以借用DQN中的很多方式来改进Critic的训练，比如用目标网络缓解自举造成的偏差等。但要注意到Critic网络和DQN中的网络存在一定的差异性（尽管从表面上看它们的网络结构相同）：

智能算法及应用@华南理工大学2025

- Critic网络是对动作价值函数 $Q_{\pi}(s, a)$ 的近似。而DQN网络则是对最优动作价值函数 $Q^*(s, a)$ 的近似。
- 对Critic网络网络的训练使用的是 SARSA 算法, 它属于On-Policy, 不能用经验回放。对 DQN 的训练使用的是 Q 学习算法, 它属于Off-Policy, 可以用经验回放。

同步优势 Actor-Critic (Synchronous Advantage Actor-Critic, A2C)

A2C算法在 Actor-Critic 算法的基础上做了一些改进, 归纳如下:

(1) A2C 算法采用优势函数 (Advantage) 作为策略梯度公式中的后一项, 也就是说 A2C 算法采用了 Baseline 标准化操作:

$$A(s_t, a_t) = Q_{\mu}(s_t, a_t) - V_w(s_t)$$

同步优势 Actor-Critic (Synchronous Advantage Actor-Critic, A2C)

实际上, 我们并不需要两个单独的网络, 因为 Q 网络 和 V 网络接收的参数都是状态 s , 再加上我们知道 Q 值和 V 值存在以下关系:

$$Q(s_t, a_t) = \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t)} [r_t + \gamma V(s_{t+1})]$$

智能算法及应用@华南理工大学2025
用单步采样近似估计有

$$Q(s_t, a_t) \approx r_t + \gamma V(s_{t+1})$$

所以我们可以使用和对应的 V 值来计算优势函数, 即:

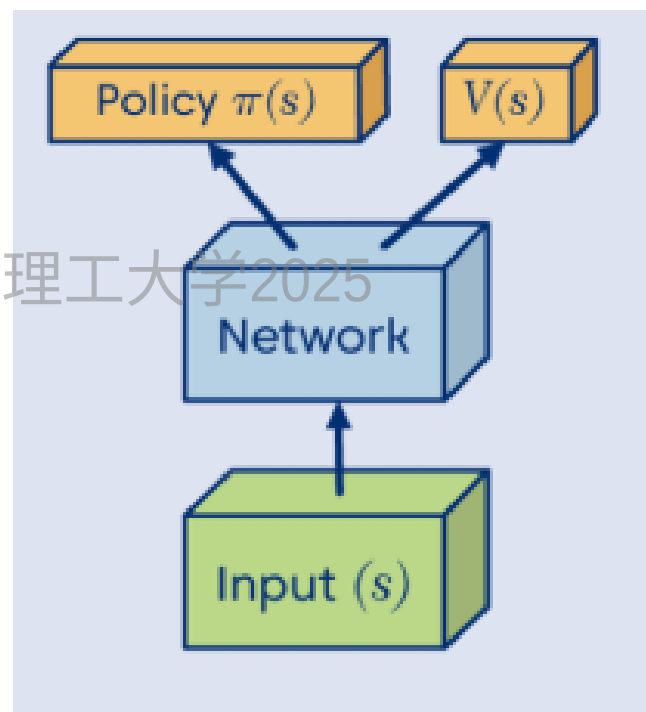
$$A(s_t, a_t) = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$$

其中 V_w 为 critic 网络。

同步优势 Actor-Critic (Synchronous Advantage Actor-Critic, A2C)

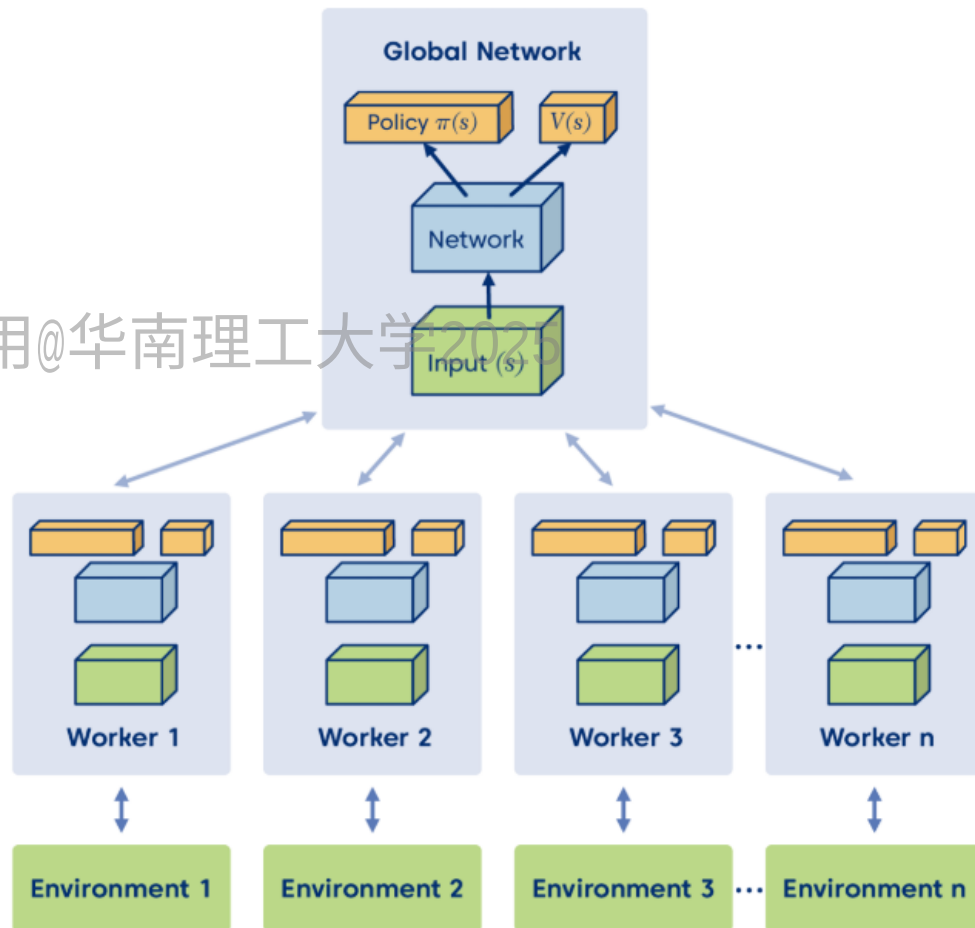
(2) A2C 算法采用了 Actor 和 Critic 共享网络参数的方式, 减少了需要学习的网络参数, 同时使得训练更稳定。具体的做法是使用同一个网络接收状态, 再经过不同的出口同时输出 π 和 V 。

(3) 在 Actor-Critic 算法的基础上增加了并行计算的设计。



异步优势 Actor-Critic (Asynchronous Advantage Actor-Critic, A3C)

- A3C有一个 master节点部署global network, 它包含 policy 的部分和 value 的部分。
- A3C同时运行多个worker节点, 每一个 worker 工作前都会把 global network 的参数复制过来。然后worker跟环境做互动, 每一个 actor 去跟环境做互动的时候, 收集到比较多样性的数据。互动完之后, 就会计算出梯度, 回传梯度给master上的global network。然后 master就会拿这个梯度去更新原来的参数。



异步优势 Actor-Critic (Asynchronous Advantage Actor-Critic, A3C)

- 注意到所有的 actor 都是并行跑的，每一个 actor 各做各的，不管彼此。所以当在一个 worker 做完想要把参数传回去的时候，本来它本来复制的 global network 参数是 θ_1 ，等它要把梯度传回去的时候。可能别人已经把原来的参数更新为 θ_2 了。但是没有关系，它一样会把这个梯度更新回去。这个就是 A3C 中的“Asynchronous”。

